ResiBFT: Scalable Byzantine Fault Tolerance for Heterogeneous Networks Under Attacks

Author 1
NDSS 2027
ndss2027.com

Author 3
NDSS 2027
ndss2027.com

Author 5
NDSS 2027
ndss2027.com

Author 7
NDSS 2027
ndss2027.com

Author 2
NDSS 2027
ndss2027.com

Author 4
NDSS 2027
ndss2027.com

Author 6
NDSS 2027
ndss2027.com

Author 8
NDSS 2027
ndss2027.com

***Abstract*—Trusted execution environment (TEE) provides substantial performance gains for Byzantine fault tolerance (BFT) protocols. However, most state-of-the-art designs suffer from an all-or-nothing limitation — they typically assume homogeneous settings in which the system consists entirely of either TEE-equipped or non-TEE replicas — an assumption undermined by physical attacks. In this paper, we address this gap by proposing the physical-resilient (PR) fault model, which explicitly captures TEE-compromised replicas capable of active equivocation. We present ResiBFT, a novel hybrid BFT protocol for heterogeneous settings containing both types of replicas, which achieves high performance while preserving safety and liveness. ResiBFT forms an adaptive committee composed exclusively of TEE-equipped replicas to support TEE-accelerated agreement along the rapid execution path. The committee-based agreement may suffer from safety or liveness violations, which can be detected and confirmed through lightweight verification. Upon detecting such failures, it automatically falls back to the common execution path, thereby ensuring continuous system operation. We prove that ResiBFT guarantees deterministic hybrid safety and fork-free path transitions, even under adversarial committee compromise. A rigorous performance evaluation spanning theoretical analysis to deployment experiments in production network proves our contributions. The results show that ResiBFT achieves high performance comparable to state-of-the-art protocols while requiring $1/3$ of the replicas to be TEE-equipped, even under targeted attacks in real-world heterogeneous environments.**

I. Introduction

Byzantine fault tolerance (BFT) serves as a cornerstone of modern distributed systems, enabling servers to provide correct service even when a subset of replicas exhibits arbitrary behaviors due to hardware failures, software crashes, or malicious attacks, known as Byzantine faults [46]. Such behaviors pose serious threats to the confidentiality, integrity, and availability (CIA) of the system. BFT systems have been adopted in cloud platforms, large-scale data centers [76], blockchain infrastructures [57], [79], and decentralized applications [5], [56] in recent years. State machine replication (SMR) is a widely used technique for implementing BFT systems. The core idea is to model the target service as a state machine and replicate it across multiple servers, which execute a totally ordered stream of events and produce identical state transitions.

Classical protocols tolerate f Byzantine replicas by deploying at least $3f+1$ replicas. Their efficiency has been improved through several lines of work, including robust attack-resilient optimizations [1], [7], committee-based agreement [69], [78], and optimistic designs [16], [38]. Introducing trusted execution environment (TEE) — such as TrustZone [58], Intel SGX [24], AMD SEV [42], and Keystone [47] — has emerged as a powerful primitive to optimize BFT protocols. By leveraging the non-equivocation property provided by TEE and the transferability of digital signatures [81], these *TEE-based* protocols reduce the replica requirement from $3f+1$ to $2f+1$ [75]. Such designs are particularly attractive for high-performance blockchains [17], [50], [62], [65] and SMR systems [26], [73], as they improve efficiency by streamlining agreement rounds [8], [23], [27], [72], eliminating redundant communication [19], [31], [48], [51], [81], accelerating view changes [26], [81], and strengthening computational security [41], [48].

However, the high performance and safety achieved by such TEE-based protocols in *homogeneous* networks comes at the cost of increasingly untenable assumptions. 'Homogeneous' means that replicas are assumed to share the same set of capabilities and exhibit similar vulnerabilities. Recent studies have shown that TEE is vulnerable to physical attacks targeting the memory subsystem, including side-channel attacks [64], [66], fault injection attacks [25], and memory-bus interposition attacks [18]. These threats undermine the CIA guarantees of TEE by exploiting the gap between cryptographic protection

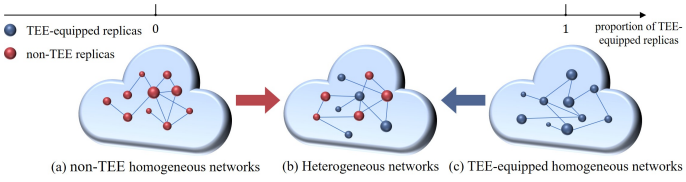


Fig. 1. The motivation for proposing ResiBFT.

Network and Distributed System Security (NDSS) Symposium 2025
24-28 February 2025, San Diego, CA, USA
ISBN 979-8-9894372-8-3
https://dx.doi.org/10.14722/ndss.2025.[23|24]xxxx
www.ndss-symposium.org

and physical memory access semantics. As a result, an adversary may extract attestation keys, generate conflicting attestations, or manipulate trusted states using low-cost hardware (under $1000). Such capabilities break the non-equivocation property and consequently lead to safety violations, such as forks, rollbacks, and double-spending. Besides, the all-TEE assumption may incur high economic costs, vendor lock-in, and substantial operational rigidity in heterogeneous networks or incremental migration environments [35], thereby limiting the practical adoption of TEE in real-world systems.

To address the lack of fault detection and crash recovery schemes for TEE failures in prior works, recent efforts have begun to explore possible solutions under a *hybrid* Byzantine fault model. Pallas [28] proposes the rollback-resilient TEE-based BFT framework that tolerates Byzantine faults and unbounded TEE crashes through an efficient view synchronizer, Aegis. TEEMS [29] introduces the restart-rollback (RR) fault model, which captures a TEE-specific failure mode that a replica may restart and rollback to an earlier version, thus lying between crash faults and Byzantine faults. MRaft [35] adopts the mixed fault (MFT) model to support partial participation of TEE-equipped replicas under the crash-only assumption. Nevertheless, these studies mainly address passive faults under predominantly TEE-based deployments and still assume the non-equivocation property. They leave open a critical gap in handling active physical attacks, which can enable equivocation and undermine agreement in homogeneous networks, as illustrated in Figure 1(a). Accommodating replicas subject to physical attacks therefore requires extending homogeneous settings to heteromorphic settings and developing a feasible BFT protocol under this model, as illustrated in Figure 1(b).

In this paper, we challenge these assumptions, considering the following fundamental research questions:
Can we retain the performance benefits of TEE while guaranteeing strong safety and liveness without relying on homogeneous networks, even when a subset of TEE-equipped replicas suffers active physical compromise?

To address this question affirmatively, we first extend the standard Byzantine fault model by introducing the physical-resilient (PR) fault model, which explicitly captures TEE-compromised replicas by their attack capabilities. Under this model, we assume a powerful adversary that can compromise TEE-equipped replicas and violate the conventional CIA assumptions, but only within a bounded attack budget.

We then propose ResiBFT, a novel hybrid BFT protocol that decouples performance optimization from strong trust assumptions. We treat TEE outputs as optimistic hints rather than traditional authoritative decisions. Building on this key insight, ResiBFT prioritizes confining proposal generation to a small, dynamic, and randomly selected committee of TEE-equipped replicas formed via verifiable random functions (VRF). All replicas are then permitted to participate as lightweight validators. The committee is active only on the *rapid* execution path, where it produces certificates for agreed client requests. If validation failures exceed a threshold, the system falls back to the *common* execution path. Our optimistic design achieves high performance close to that of state-of-the-art protocols in wide-area networks (WAN) while requiring significantly lower TEE coverage, and provides deterministic degradation when TEE is compromised or validation fails.

This paper presents the following key contributions:

- **Physical-resilient fault model.** We formalize the hybrid physical-resilient (PR) fault model that extends the standard Byzantine fault model with TEE-compromised replicas to capture active physical attacks, thereby bridging the gap between theoretical assumptions and real-world adversarial capabilities.
- **ResiBFT protocol.** We present ResiBFT, a hybrid BFT protocol for heterogeneous networks under the physical-resilient (PR) fault model, highlighting efficient agreement with minimal reliance on TEE. ResiBFT reaches agreement efficiently along the rapid execution path and switches to the common execution path when necessary. We prove that it preserves safety and liveness through lightweight verification, checkpoint consistency, fault detection, and crash recovery, thereby ensuring fork-free transitions and seamless path switching.
- **Implementation and Evaluation.** We implement a prototype of ResiBFT and evaluate performance and attack resilience against state-of-the-art protocols in WAN. The results show that ResiBFT achieves the best performance while requiring $1/3$ of the replicas to be TEE-equipped, demonstrating a practical approach toward deployable TEE-based agreement in heterogeneous networks.

The remaining sections are organized as follows. Section II reviews related works. Section III defines physical-resilient (PR) fault model. Section IV presents an overview of ResiBFT. Section V provides key mechanisms and fault detections. Section VI details the implementation. Section VII analyzes safety and liveness. Section VIII reports experimental results, and finally, Section IX concludes our paper.

II. Related Works

BFT protocols must satisfy the standard correctness criteria of *safety* and *liveness*. 'Safety' guarantees that any two replicas remain mutually consistent, while 'liveness' ensures that replicas eventually reach consensus on requests [63]. However, to circumvent the Fischer-Lynch-Paterson (FLP) impossibility result [34], researchers extend the basic model in several ways, most notably through randomization [6], [44], [55] and by adding timing assumptions [70]. In particular, BFT systems commonly adopt a partial synchrony model [33], under which they have addressed challenges such as high communication complexity, limited scalability, and restricted applicability. Among them, PBFT [13] marked a milestone by introducing the first practical and efficient BFT protocol and inspiring a large body of subsequent work.

A. Physical Attacks

Physical attacks on TEE exploit the gap between protected on-chip execution and untrusted off-chip hardware components. The adversary need not directly compromise TEE itself,

but can leverage weaknesses in off-chip memory protection instead to recover secrets, induce incorrect memory mappings, replay stale data, or even compromise attestation keys. Specifically, side-channel attacks infer sensitive information from observable leakage, including access patterns and memory traffic features. Memory-bus interposition attacks insert malicious devices into the memory channel to monitor or manipulate data transfers. Fault injection attacks deliberately perturb external signals to corrupt memory accesses or execution states. These results show that memory encryption alone is insufficient to guarantee TEE security when confidentiality, integrity, freshness, and physical tamper resistance are weak or absent [37], [42]. To the best of our knowledge, no existing BFT system is specifically designed to defend against physical attacks. This gap motivates the design of ResiBFT.

B. Hybrid BFT Protocols

Hybrid BFT protocols integrate additional components or assumptions for purposes such as attack resilience [29], [74], fault recovery [12], [31], performance optimization [26], [27], and broaden fault tolerance [52], [60]. 'Hybrid' refers to the combination of multiple subsystems operating under different sets of assumptions [30], [68], [70].

TEE-based protocols. TEE-based protocols assume that the trusted components either work according to specification or fail by crashing, even when the host or virtual environment is compromised. They must balance a compact trusted computing base (TCB) against high performance. [23] achieves an atomic multicast protocol using trusted timely computing base (TTCB), while [19], [49] implement a trusted log based on attested append-only memory (A2M). [48] further shows that trusted components need only provide monotonic counters to ensure non-equivocation by TrInc, an idea later adopted by [39], [41], [51], [72], [73] through various trusted subsystems. Leveraging Intel SGX, Hybster [8] enables parallel instance execution using TrInX, Damysus [26] defines the CHECKER and ACCUMULATOR services to improve performance over HotStuff [80], and OneShot [27] further reduces the number of agreement rounds through an optimistic design. TBFT [81] limits message complexity to linear by lightweight cryptographic primitives. [82] proposes a leaderless protocol based on trusted logs and monotonic counters. However, these assumptions become difficult to sustain in heterogeneous settings or under active physical attacks.

Alternative fault models. [28], [29], [35] extend the standard Byzantine fault model with additional assumptions to tolerate a wider range of failures. UpRight [20] adopts a hybrid fault model with separate upper bounds for crash faults and Byzantine behaviors. VFT-SMaRt [60] proposes the Visigoth fault tolerance model (VFT), which separately parameterizes arbitrary faults, collusion failures, and slow replicas. XPaxos [52] relies on the cross fault tolerance model (XFT), which requires a single fault threshold to jointly combine Byzantine behaviors, crash faults, and partitioned replicas. SplitBFT [54] treats each replica not as a single unit of failure, but as a composition of failure-isolated PREPARATION, CONFIRMATION, and EXECUTION compartments. Algorand [36] and Pando [78] introduce a near-optimal resilience model with a slightly more restrictive bound on the number of Byzantine replicas.

C. Robust Protocols Under Attacks

BFT systems may suffer severe performance degradation under physical attacks, Denial-of-Service (DoS) attacks, or performance attacks. Spinning [71] and Aardvark [21] mitigate performance attacks through periodic leader replacement, resource isolation, and request filtering. ROTE [53] implements rollback protection as a distributed integrity service across collaborating TEE on separate processors, allowing a restarted replica to recover a fresh state. Eges [16] reduces the impact of DDoS attacks through a stealth committee, which effectively conceals the real committee within a large pool of fake replicas. FlexiTrust [40] reverts to the $3f+1$-replica configuration in to support parallel execution while preserving safety under rollback attacks in TEE-equipped homogeneous networks.

D. Classical Optimizations

Speculative execution. The strict requirement that all honest replicas execute requests in the order determined in the agreement stage can significantly limit system performance. To address this issue, prior works adopt speculative execution [20], [39], [43], [73] or tentative execution [13], [16], [36], allowing replicas to relax the execution order. Such techniques typically rely on a rollback mechanism that enables a replica to revert to a consistent state when correct replicas diverge due to inconsistent execution orders.

Committee-based protocols. In WAN settings, involving all replicas in agreement may degrade system performance. [4], [10], [16], [36], [59], [69], [78] solve this problem by fairly electing a committee from a large replica pool and delegating agreement to it. [16] adopts a trusted pseudo-random number generator as the randomness source for establishing the stealth committee. [36], [69], [78] use the seed-generation algorithms with VRF to dynamically select a public committee and support committee rotation.

Optimistic protocols. Since BFT systems operate normally most of the time, [16], [27], [31], [32], [41], [51] provide a gracious execution path under optimistic conditions and fall back to a normal execution path when failures are detected, thereby maximizing performance.

Such optimizations largely operates in homogeneous networks, with limited exploration of heterogeneous networks.

III. PHYSICAL-RESILIENT FAULT MODEL

A. Heterogeneous Networks

For simplicity, we refer to replicas without TEE and replicas equipped with TEE as *G-Replicas* and *T-Replicas*, respectively. Heterogeneous networks contain both of them. Each replica maintains static context, including local states, state-transition functions, message queues, timers, and logs. As depicted in Figure 2, the static context of G-Replicas and T-Replicas is maintained in untrusted parts and trusted parts, respectively. Messages are transmitted over unreliable channels. Although

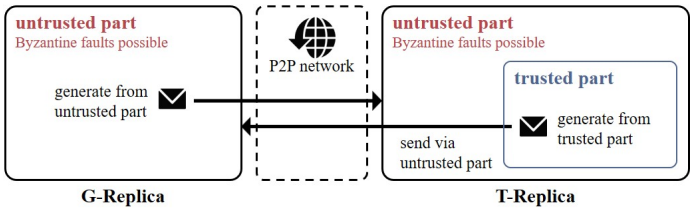


Fig. 2. The hybrid model in heterogeneous networks.

real-world deployments may involve TEE with different models and capabilities [54], we assume that they provide uniform guarantees (S1) – (S3):

(S1) **Integrity protection.** The integrity is protected through encrypted storage, integrity verification, and remote attestation [22], enabling replicas to cryptographically attest to TEE states and integrity of stored data to external entities while resisting replay attacks and impersonation attacks.

(S2) **Confidentiality protection.** Sensitive code and data can be securely accessed and processed in an isolated environment called *enclave*. Operating systems, virtual platforms, and user-level programs cannot directly access or modify data inside enclaves, reducing the risk of data leakage. Each T-Replica incorporates a non-volatile monotonic counter to prevent rollback attacks [53], [61], [77].

(S3) **Availability protection.** TEE can still protect data storage and code execution even when deployed on a compromised or attacked host, or in a malicious or privileged software environment, thereby supporting critical tasks.

B. System Model

[39] prove that the system requires at least $3f+1$ replicas when fewer than $2f+1$ T-Replicas are available. W.l.o.g., we consider a system with $n = 3f+1$ replicas in heterogeneous networks, where f denotes the number of Byzantine replicas. They are uniquely indexed as $\mathcal{R}_i, i = 1, 2, \cdots, n$, and each identifier is known to all replicas. All T-Replicas follow the same (S1) – (S3). We define a class of *TEE-compromised* replicas within f Byzantine replicas, denoted by $f_C = \lambda f$, where λ is a small constant bounded by a limited attack budget. 'TEE-compromised replica' refers to a T-Replica with a physically compromised enclave under partial adversarial control.

All replicas employ standard cryptographic primitives, including secure hash functions $\mathcal{H}$ for sequence compression, threshold signature schemes [9], [11], [67] for message authentication, and VRF for committee selection. $\mathcal{H}$ provides collision resistance and the one-way property under the computational boundedness assumptions. For a sequence x, we define its cryptographic identifier as digest $\mathcal{H}(x)$. We assume a secure public key infrastructure (PKI) and a key distribution protocol that provide the keys required for message authentication and committee selection to all intended parties.

Remote attestation allows a T-Replica to prove its properties to external entities by providing verifiable evidence. It multicasts an attestation containing its TEE configuration to the other replicas. We assume that only T-Replicas can mutually authenticate one another through remote attestation [3]. The authenticity of such attestations is ensured by certificate authorities (CA) and hardware vendors.

We adopt a standard communication model used by most BFT systems. We assume that all replicas are deployed in a peer-to-peer network based on partial synchrony model [33] with a known bound Δ and an unknown Global Stabilization Time (GST). Messages may not arrive before GST [34]. After GST, communication between active replicas will arrive in Δ. Communication channels are unreliable — messages may be duplicated, dropped, delayed, or reordered — but retransmitted messages are eventually delivered.

C. Threat Model

We consider the physical-resilient (PR) fault model, a hybrid model that combines Byzantine faults with crash-recovery failures while explicitly accounting for TEE-compromised replicas. A replica is honest (or non-faulty) if it follows its specification, except that it may crash and later recover. They respond to messages within Δ after GST, participate in reaching agreement only for valid requests, and commit requests correctly. Otherwise, a replica is Byzantine (or malicious) if it may behave arbitrarily.

We assume a strong adversary that can coordinate Byzantine replicas to compromise our system. The attack vectors by the adversary over a replica depends on the type of its host.

- **G-Replicas.** The adversary may compromise the code, data, or states, enabling it to support for null, duplicate, conflicting, or corrupted requests and to disrupt agreement by dropping, reordering, or ignoring messages.
- **T-Replicas.** The adversary may manipulate I/O operations at the TEE boundary or forcibly abort TEE execution. Under the computational boundedness assumptions, malicious T-Replicas can only exhibit observable unresponsive behaviors, since the adversary cannot generate cryptographically valid responses.

TEE-compromised replicas can be viewed as an intermediate class between Byzantine G-Replicas and Byzantine T-Replicas. They are subject to the following (V1) – (V3):

(V1) **Active equivocation.** The adversary can bypass (S1) and use intercepted private keys and sensitive data to generate conflicting messages through equivocation, thereby violating safety guarantees and leading to forks, rollbacks, and double-spending.

(V2) **Data leakage.** In general, the adversary cannot arbitrarily modify data inside the enclave under (S2), but may still tamper with observed data by interfering with off-chip memory, the memory bus, or external signals.

(V3) **Denial of service.** If the adversary cannot directly extract or tamper with data, it may still render TEE unable to satisfy (S3) through physical interference and thus violate liveness guarantees.

The implementation difficulty of these attack vectors on TEE-compromised replicas is ordered as (V1) > (V2) > (V3). We assume that the adversary operates under a limited attack budget, such that only a small number of f_C replicas can be

TEE-compromised. TEE-compromised replicas are not fully equivalent to the attack capabilities of Byzantine G-Replicas under (V1) – (V3). They are restricted to only partial malicious behaviors within the enclave, but cannot control host-level networking or forge messages sent by untrusted parts.

The adversary selects attack vectors strategically to obstruct agreement progress or maximize consensus delay while minimizing the risk of detection. However, under the computational boundedness assumptions, it cannot subvert fundamental cryptographic primitives. It also cannot delay messages exchanged by honest replicas. In the worst case, the adversary fully compromises all f Byzantine replicas, obtains complete knowledge of their static context and private keys, and launches (V1) against TEE-compromised replicas. However, the adversary is unable to access the local states of T-Replicas (except for TEE-compromised replicas) protected by intact TEE.

A replica is considered as crashed (or unresponsive) when no honest replica receives its messages within Δ after GST. Possible causes include replica failures or disconnections, a Byzantine G-Replica pretending to be offline or crashed, a TEE-compromised replica violating (V3), and a Byzantine T-Replica maliciously delaying its responses.

IV. ResiBFT Overview

We present an overview of ResiBFT, which achieves both robustness and efficiency in heterogeneous networks. 'Robustness' refers to satisfying both safety and liveness, while 'efficiency' is reflected in lower latency than in state-of-the-art protocols. ResiBFT exploits the non-equivocation property of T-Replicas while tolerating potentially malicious G-Replicas. To this end, it combines the efficiency of TEE-based protocols with the robustness of classical protocols through an optimistic design and adopts the committee selection and revocation to switch between the rapid path and the common path.

Committee-based agreement is widely used in BFT systems and blockchains. Permissioned blockchains typically run their protocols over a static and explicit committee. However, such static committees are vulnerable to DoS attacks and network partition attacks in WAN. Thus, ResiBFT periodically selects several T-Replicas to form a committee $\{\mathcal{R}_C\}$ according to a predefined committee election algorithm [36]. C is a specific list that map to i. When a committee is active, we refer to its members as *internal* replicas and to the remaining replicas as *external* replicas. Let k denote committee size ($|\{\mathcal{R}_C\}| = k$). Globally, k may vary dynamically with committee election and revocation. Once a committee has been elected and confirmed, k remains fixed until it is replaced.

ResiBFT adopts HotStuff [80] as the agreement protocol on the common path and Damysus [26] on the rapid path. Both are representative works with publicly available implementation details. The intuition of ResiBFT is to have internal replicas execute Damysus along the rapid path to reach local agreement, with the leader then multicasting a certificate for the agreed block to all replicas for global lightweight verification. 'Local' refers to the committee level, whereas 'global' refers to the system-wide level. If sufficient replicas successfully verifies the certificate, the system executes the block and moves to the next view along the same path. Otherwise, if the number of verification failures exceeds a predefined threshold, the system falls back to the common path safely. To preserve a fork-free transition between these two paths, each replica maintains a stable checkpoint. This hybrid design enables ResiBFT to leverage the stronger safety and higher efficiency of T-Replicas without requiring full TEE deployment, thereby reducing the cost and compatibility challenges of large-scale BFT systems.

Periodic agreement. We define *view* v as the basic unit of agreement. As shown in Figure 3, a view follows the same path as the previous one unless specific conditions trigger a path transition. To describe this periodic behaviors, we define *epoch* e as a sequence of consecutive views that contains two path transitions — common path $\rightarrow$ rapid path $\rightarrow$ common path. Accordingly, the overall execution can be modeled as an infinite sequence of repeating epochs. To describe the consensus process over time, each epoch is divided into Normal stage, Gracious stage, and Uncivil stage according to the corresponding path switches. Replicas exhibit the same behavior within the same phase.

Identities management. With the support of remote attestation, a T-Replica can authenticate other T-Replicas and place them in the *T-Set*, while assigning the remaining replicas to the *G-Set*. We denote these sets by $\{\mathcal{R}_T\}$ and $\{\mathcal{R}_G\} = \{\mathcal{R}_i\} \setminus \{\mathcal{R}_T\}$, whose cardinalities are referred to as $|\{\mathcal{R}_T\}| = d$ and $|\{\mathcal{R}_G\}| = n - d$, respectively. T and G are specific lists that map to i. Replicas in G-Set are untrusted and are therefore ineligible for committee election. If a T-Replica fails to provide a valid integrity attestation, it is placed in G-Set. We assume that TEE-compromised replicas can still participate in remote

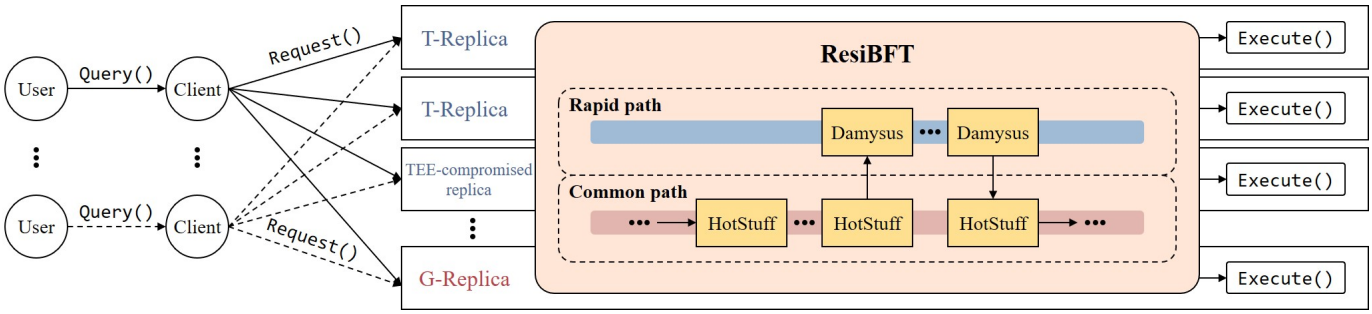


Fig. 3. The system architecture of ResiBFT.

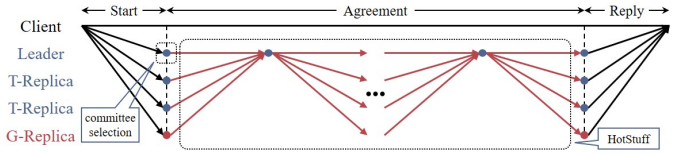


Fig. 4. The message patterns in NORMAL stage and UNCIVIL stage.

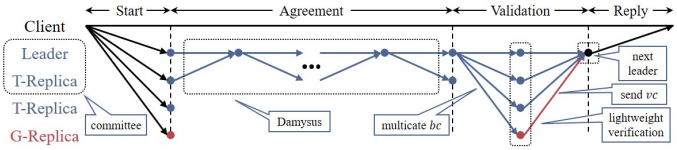


Fig. 5. The message patterns in GRACIOUS stage.

attestation on track. Otherwise, they would also be placed in G-set and would be observable only as G-Replicas. All replicas in T-Set maintain a consistent membership view of $\{\mathcal{R}_T\}$. G-Replicas do not know the members in T-Set and G-Set.

Leader rotation. ResiBFT follows a 'streamlined' protocol in which the leader rotates in each view [14], [15]. This design improves both security and performance by eliminating the communication overhead of a view-change protocol.

HotStuff. & Damysus. HotStuff employs a 3-phase agreement, including PREPARE phase, PRE-COMMIT phase, and COMMIT phase. A replica accepts and votes for a proposal only if it satisfies SAFENODE predicate [80]. To guarantee safety and liveness, HotStuff employs a *locking* scheme. A replica is locked on the voted block in COMMIT phase and subsequently votes for blocks not less than the locked block. Damysus improves upon HotStuff by reducing to a 2-phase agreement, as PREPARE phase and PRE-COMMIT phase. To preserve safety and liveness without relying on the locking scheme, it designs CHECKER and ACCUMULATOR. CHECKER guarantees the valid block commitment and correct proposal verification by maintaining a non-volatile trusted monotonic counter. ACCUMULATOR, which is only called by the leader, identifies the highest block among a given set of blocks.

V. RESIBFT DESIGN

We present the design mechanisms following the progression of ResiBFT over time. The transitions between NORMAL stage, GRACIOUS stage, and UNCIVIL stage are designed to preserve monotonic views and maintain consistent states.

A. NORMAL *stage*

A new epoch begins in NORMAL stage, during which the system reaches agreement on the common path by executing HotStuff, as presented in Figure 4. They proceed by attempting to propose and form a committee. A leader in G-Set cannot propose a committee since T-Set are unknown to it. When the leader rotates to a replica in T-Set, it proposes a committee by a predefined committee election algorithm. After the committee is formed, the system transitions from the common path to the rapid path and enters GRACIOUS stage.

We use VRF as the committee election algorithm. Each $\mathcal{R}_i$ maintains a pair of keys $(Pub_i, Priv_i)$. Given a sequence x derived from a shared seed generation algorithm among all replicas, VRF outputs a pseudo-random sequence $V(x)$ and its proof $\pi(x)$ by $Priv_i$. They use the hash of the block committed lb views before the current view as the seed, where lb (look-back) is a system parameter chosen to be large enough to ensure liveness. Replicas whose $V(x)$ values rank among the lowest k are elected to the committee. k is randomly sampled from a predefined interval, thereby determining the size of the committee. To provide strong safety and liveness guarantees, we denote by $\underline{k}$ the minimum committee size required to tolerate dozens of Byzantine and TEE-compromised replicas [38]. All replicas can verify the elected committee by Pub_i. The randomized and unpredictable selection process prevents an adaptive adversary from manipulating the committee composition in advance. The elected committee members then load Damysus into their TEE in preparation for the rapid path.

B. GRACIOUS *stage*

GRACIOUS stage begins once a committee $\{\mathcal{R}_C\}$ has been proposed, selected, and recorded by all replicas. In this stage, internal replicas remain 'active' and execute Damysus, while external replicas remain 'passive' and participate lightweight verification and final agreement.

Each replica records the latest committed block as a stable *checkpoint* cp in a view of GRACIOUS stage. Once internal replicas reach local agreement on a block, the leader multicasts the agreement evidence to all replicas in the form of a *block certificate* bc. Each replica then generates a *validation certificate* vc through lightweight verification, which serves as a cryptographic proof of bc. A vc can take one of three states — 'accepted', 'rejected', or 'timeout'. ResiBFT employs a cross-validation to preserve safety, allowing the next leader to take appropriate actions based on at least $2f+1$ vcs collected from the previous view. As presented in Figure 5, both internal and external replicas send their vcs after lightweight verification to the next leader. If a replica generates an accepted-vc, it tentatively executes the block without committing it to the clients. The next leader proceeds with agreement only after collecting at least $f+1$ accepted-vcs. It then multicasts these vcs, instructing all replicas to commit the previous block to the clients and update cp. Each replica maintains only a single cp at any given time for state recovery.

C. UNCIVIL *stage*

On the rapid path, once a committee failure is detected (see Section V-D), honest replicas declare the failure by generating rejected-vcs or timeout-vcs. The system falls back to the common path and completes committee revocation after the leader collecting at least $f+1$ rejected-vcs or timeout-vcs, enabling the system to transition from GRACIOUS stage to UNCIVIL stage. Byzantine leaders cannot forge such a collection of vcs. All replicas then reach agreement by HotStuff. New proposals must extend from the latest cp to prevent forks during the transition. Byzantine replicas may delay messages to obstruct committee revocation, which may take multiple views to complete. We introduce UNCIVIL stage to buffer the

intermediate views during a complete transition. After that, all replicas end the current epoch and move on to the next one.

D. Fault Detection and Recovery

Committee-based agreement can improve the efficiency of ResiBFT, but it may compromise local safety and liveness. Therefore, ResiBFT adopts a passive fault-recovery approach [13], [45], [52] based on fault detection [7], [16], [73] when either the committee or the leader fails. Specifically, ResiBFT detects failures through timeout events raised by two timers and through lightweight verification supported by all replicas, thereby preventing potential safety and liveness violations [2].

We assume each replica is equipped with two physical security timers, $\mathrm{A_v}$ and $\mathrm{A_e}$, to monitor the time cost of agreement process at the view and epoch levels. 'Epoch-level' and 'view-level' indicate that $\mathrm{A_e}$ and $\mathrm{A_v}$ can be triggered at most once per epoch and per view, respectively. The corresponding upper bounds are defined by thresholds α_v and $\alpha_\mathrm{e} = \mu\alpha_\mathrm{v}$, where μ is an adaptive variable. An alert notifies the replica when either timer expires. When replicas reach consensus on a block in a view, both timers are reset.

Committee failures. At any time, a committee may fails due to the unresponsive T-Replicas and TEE-compromised replicas, mainly for the following (R1) – (R2) in liveness and (R3) in safety:

(R1) **Benign timeout with a honest leader.** An honest leader may be unable to lead the committee through prescribed agreement process (e.g., it may fail to receive sufficient votes and remain in a waiting state). Under the PR fault model, the leader cannot distinguish the cause of unresponsive behaviors — they may be deliberately disguised by Byzantine T-Replicas and TEE-compromised replicas, or may simply stem from temporary crashes.

(R2) **Malicious timeout with a Byzantine leader.** The adversary may control more than half, or even all, of the internal replicas, including the Byzantine leader. If the number of TEE-compromised replicas is insufficient to force a malicious, incorrect or manipulated committee decision, they may still collude to behave as unresponsive replicas, thereby obstructing agreement process and making the committee appear persistently unresponsive.

ResiBFT guarantee liveness by the epoch-level timer $\mathrm{A_e}$. The timer starts from the previous successful agreement. If an external replica fails to receive a bc within α_e, it detects an observable committee failure by triggering an alert via $\mathrm{A_e}$. Honest replicas send timeout-vcs to the next leader. Once at least $f+1$ timeout-vcs have been collected, the system transitions to UNCIVIL stage.

(R3) **Manipulated committee agreement.** When the number of TEE-compromised replicas exceeds half of the internal replicas, they may vote for an invalid block. The leader can aggregate these votes into a seemingly valid bc. This indicates that the committee has reached an malicious, incorrect or manipulated local agreement.

ResiBFT guarantee safety by the cross-validation (see Section V-B). Once at least $f+1$ rejected-vcs have been collected, the system transitions to UNCIVIL stage.

Leader failures. ResiBFT guarantees liveness by the view-level timer $\mathrm{A_v}$ when a leader fails. Leader failures are mainly detected at the following points:

- **On the common path.** $\mathrm{A_v}$ expires when a replica fails to receive messages from the leader within α_v on the common path. Honest replicas then multicast TIMEOUT messages to all replicas. Once at least $f+1$ TIMEOUT messages has been collected, the new leader starts the next view by multicasting these TIMEOUT messages.
- **On the rapid path.** ResiBFT gives priority to handling leader failures within the committee on the rapid path, while external replicas remain unaware of the process. If the committee remains unable to make progress after multiple leader changes among the internal replicas, it is regarded as a committee failure. The time cost of leader changes within the committee is upper-bounded by the timescale factor μ. The process of switching leaders is similar to that on the common path.

VI. RESIBFT IMPLEMENTATION

A. Definitions and Notations

Epochs & Views. 'View' captures short-term state progression, whereas 'epoch' represents a longer timescale. Epoch e increases monotonically. View v^e_j denotes the j-th view in e, where j increases monotonically. Let B^e_j denote the block agreed upon in v^e_j, where $\mathrm{B}^\mathrm{e}_j = (\mathrm{H}, \mathrm{T})$, with $\mathrm{H} = \mathcal{H}\left(\overleftarrow{\mathrm{B}^\mathrm{e}_j}\right)$ denoting the digest of its predecessor block and T denoting the set of transactions.

Quorum sizes. ResiBFT extends HotStuff and Damysus with alternative executions. We define q as the stage-dependent quorum size. In NORMAL stage and UNCIVIL stage, q is fixed as $q_G = 2f+1$. In GRACIOUS stage, q adapts to the committee size k as $q_T = \lfloor (k+1)/2 \rfloor$.

Messages. We adopt standard notation for messages signing and verification. Sender $\mathcal{R}_i$ of message M generates a digital signature $\langle \mathrm{M} \rangle_{\sigma_i}$ by private key, which can be verified with the public key. A *quorum certificate* $qc(\mathrm{M}) = \left(\langle \mathrm{M} \rangle_{\sigma_1}, \langle \mathrm{M} \rangle_{\sigma_2}, ...\right)$ is denoted as a collection of multiple signatures. Replicas use block-view pairs (H, v) to store, transmit, and update states. We denote the current phase by ph.

B. High-level Implementation

ResiBFT inherits the agreement schemes of HotStuff and a Damysus variant, including message patterns and TEE services. T-Replicas maintain prepared blocks within TEE, while G-Replicas track locked blocks for consistency. The high-level implementation is detailed in Algorithm 1. NORMAL stage (Lines 6–15) and UNCIVIL stage (Lines 31–40) follow the message patterns in HotStuff. The agreement and validation in GRACIOUS stage (Lines 16–30) relies on the Damysus variant. `HotStuff` and `Damysus` represent the normal state-transition functions defined in the respective protocols.

Algorithm 1 High-level implementation of ResiBFT

// initialization for all replicas
epoch $\mathrm{e} \leftarrow 0$, view $\mathrm{v}_j^{\mathrm{e}} \leftarrow \mathrm{v}_0^0$
replicas $\{\mathcal{R}_i\}$, committee $\{\mathcal{R}_C\} \leftarrow \varnothing$
// initialization for T-Replicas
get $\{\mathcal{R}_T\}$ by remote attestation, $\{\mathcal{R}_G\} \leftarrow \{\mathcal{R}_i\} \setminus \{\mathcal{R}_T\}$
// NORMAL stages
▷ as T-Replicas
if I am the leader **and** $\{\mathcal{R}_C\} == \varnothing$ **then**
  select a committee $\{\mathcal{R}_\perp\}$ and multicast it
  run **HotStuff**, $\{\mathcal{R}_C\} \leftarrow \{\mathcal{R}_\perp\}$, $j \leftarrow j+1$
else
  run **HotStuff**, $\{\mathcal{R}_C\} \leftarrow \{\mathcal{R}_\perp\}$, $j \leftarrow j+1$
end
▷ as G-Replicas
run **HotStuff**, $\{\mathcal{R}_C\} \leftarrow \{\mathcal{R}_\perp\}$, $j \leftarrow j+1$
// GRACIOUS stages
▷ as T-Replicas
if I am the committee member **then**
  if I am the leader **and** check accepted-vcs, **then**
    run **Damysus**, multicast bc, send vc, $j \leftarrow j+1$
  else
    run **Damysus**, send vc, $j \leftarrow j+1$
  end
else
  initialize checkpoint cp
  listen for bc to update cp, send vc, $j \leftarrow j+1$
end
▷ as G-Replicas
initialize checkpoint cp
listen for bc to update cp, send vc, $j \leftarrow j+1$
// UNCIVIL stages
▷ as T-Replicas
if I am the leader **and** fail to check accepted-vcs, **then**
  delete the committee
  run **HotStuff**, $\{\mathcal{R}_C\} \leftarrow \varnothing$, $\mathrm{e} \leftarrow \mathrm{e}+1$, $j \leftarrow 0$
else
  run **HotStuff**, $\{\mathcal{R}_C\} \leftarrow \varnothing$, $\mathrm{e} \leftarrow \mathrm{e}+1$, $j \leftarrow 0$
end
▷ as G-Replicas
run **HotStuff**, $\{\mathcal{R}_C\} \leftarrow \varnothing$, $\mathrm{e} \leftarrow \mathrm{e}+1$, $j \leftarrow 0$

C. Details in GRACIOUS stage

We now describe the detailed implementation of the GRACIOUS stage in this section. We assume that view $\mathrm{v}_\perp^{\mathrm{e}}$ belongs to GRACIOUS stage. Each $\mathcal{R}_i$ has sent a $vc\left(\mathrm{v}_{\perp-1}^{\mathrm{e}}\right)$ for the previous view $\mathrm{v}_{\perp-1}^{\mathrm{e}}$ and stored a checkpoint cp. The specific data structure of cp is defined later.

NEW-VIEW phase. Each internal replica $\mathcal{R}_i$ sends a $\langle$NEW VIEW$, (\mathrm{H}_\varnothing, \mathrm{v}_\perp^{\mathrm{e}}), (\mathrm{H}_v, \mathrm{v}_v), ph\rangle_{\sigma_i}$ to the leader $\mathcal{R}_{L^*}$, where $(\mathrm{H}_v, \mathrm{v}_v)$ denotes the highest prepared block and its corresponding view number. Here, L^* denotes a value specific to i. In general, a superscript * appended to a notation indicates a specific value of that notation. $\mathrm{H}_\varnothing$ represents a dummy hash.

PREPARE phase. Upon receiving q_T NEWVIEW messages from internal replicas and q_G $vc\left(\mathrm{v}_{\perp-1}^{\mathrm{e}}\right)$s from all replicas, $\mathcal{R}_{L^*}$ verifies the validation for $\mathrm{v}_{\perp-1}^{\mathrm{e}}$, selects the highest view and its corresponding hash $(\mathrm{H}_{v^*}, \mathrm{v}_{v^*})$ by ACCUMULATOR, proposes a new block $\mathrm{B}_{\perp*}^{\mathrm{e}}$, and then multicasts a $\langle$PROPOSAL, $(\mathcal{H}(\mathrm{B}_{\perp*}^{\mathrm{e}}), \mathrm{v}_\perp^{\mathrm{e}}), (\mathrm{H}_{v^*}, \mathrm{v}_{v^*}), ph, \mathrm{B}_{\perp*}^{\mathrm{e}}, qc(\text{NEWVIEW}), qc(vc(\mathrm{v}_{\perp-1}^{\mathrm{e}}))\rangle_{\sigma_{L^*}}$ to internal replicas. If $\mathcal{R}_{L^*}$ receives at least $f+1$ rejected-vcs or timeout-vcs, it instead multicasts a PROPOSAL message to all replicas by extending the block from cp. This initiates committee revocation and transitions to UNCIVIL stage. A TEE-compromised leader may deliberately propose a block at an incorrect height or continue committee agreement despite receiving sufficient rejected-vcs or timeout-vcs. In either case, the resulting incorrect bc will be detected through lightweight verification.

Each internal replica $\mathcal{R}_i$ checks the PROPOSAL message, store the proposed block $\mathrm{B}_{\perp*}^{\mathrm{e}}$, and votes for it by sending a $\langle$PREPARE$, (\mathcal{H}(\mathrm{B}_{\perp*}^{\mathrm{e}}), \mathrm{v}_\perp^{\mathrm{e}}), (\mathrm{H}_{v^*}, \mathrm{v}_{v^*}), ph\rangle_{\sigma_i}$ to $\mathcal{R}_{L^*}$.

PRE-COMMIT phase. Upon receiving q_T PREPARE messages, $\mathcal{R}_{L^*}$ verifies them, marks $\mathrm{B}_{\perp*}^{\mathrm{e}}$ as prepared block, and multicasts a $\langle qc(\text{PREPARE})\rangle_{\sigma_{L^*}}$ to internal replicas.

Each internal replica $\mathcal{R}_i$ checks the qc(PREPARE), marks proposed $\mathrm{B}_{\perp*}^{\mathrm{e}}$ as prepared block, and votes for it by sending a $\langle$PRECOMMIT$, (\mathcal{H}(\mathrm{B}_{\perp*}^{\mathrm{e}}), \mathrm{v}_\perp^{\mathrm{e}}), ph\rangle_{\sigma_i}$ to $\mathcal{R}_{L^*}$.

DECIDE phase. Upon receiving q_T PRECOMMIT messages, $\mathcal{R}_{L^*}$ verifies them, tentatively executes $\mathrm{B}_{\perp*}^{\mathrm{e}}$, multicasts a $\langle qc(\text{PRECOMMIT})\rangle_{\sigma_{L^*}}$ to internal replicas, and multicasts a bc to all replicas for lightweight verification.

Each internal replica $\mathcal{R}_i$ checks the qc(PRECOMMIT), and tentatively executes $\mathrm{B}_{\perp*}^{\mathrm{e}}$.

VALIDATION phase. We define bc as $bc = \{\mathrm{B}_{\perp*}^{\mathrm{e}}, qc(\text{PRE COMMIT}), qc\left(vc\left(\mathrm{v}_{\perp-1}^{\mathrm{e}}\right)\right)\}$, where $\mathrm{B}_{\perp*}^{\mathrm{e}}$ denotes the tentatively agreed block, qc(PRECOMMIT) denotes the latest voting qc, $qc\left(vc\left(\mathrm{v}_{\perp-1}^{\mathrm{e}}\right)\right)$ denotes the vcs collected from q_G $vc\left(\mathrm{v}_{\perp-1}^{\mathrm{e}}\right)$s. This means that the previous block $\mathrm{B}_{\perp-1}^{\mathrm{e}}$ is confirmed only when $qc\left(vc\left(\mathrm{v}_{\perp-1}^{\mathrm{e}}\right)\right)$ contains more than $f+1$ accepted-vcs. In other words, T-Replicas have marked $\mathrm{B}_{\perp-1^*}^{\mathrm{e}}$ as prepared, while G-Replicas have locked on it in view $\mathrm{v}_{\perp-1}^{\mathrm{e}}$. Upon receiving bc, all replicas commit $\mathrm{B}_{\perp-1}^{\mathrm{e}}$, after which T-Replicas and G-Replicas validate $\mathrm{B}_{\perp*}^{\mathrm{e}}$ using CHECKER and SAFENODE predicate, respectively. Each replica $\mathcal{R}_i$ sends either an accepted-$vc(\mathrm{v}_\perp^{\mathrm{e}})$ or a rejected-$vc(\mathrm{v}_\perp^{\mathrm{e}})$ to the next leader according to its validation result. Finally, all replicas update cp, defined as $cp = \left\{\mathrm{B}_{\perp-1}^{\mathrm{e}}, \mathrm{v}_{\perp-1}^{\mathrm{e}}, qc(\text{PRECOMMIT}), qc\left(vc\left(\mathrm{v}_{\perp-1}^{\mathrm{e}}\right)\right)\right\}$, which serves as the recovery checkpoint for the next committee revocation.

VII. ANALYSIS

ResiBFT tolerates the local safety and liveness violations, which can be handled through lightweight verification and timeout alerts. Therefore, it is sufficient to rigorously prove only global safety and liveness.

A. Global Safety and Liveness

We define any two blocks B_1 and B_2 as conflicting blocks when none of these conditions $\mathrm{B}_1 \leftarrow \mathrm{B}_2$, $\mathrm{B}_2 \leftarrow \mathrm{B}_1$, or $\mathrm{B}_1 =$

B_2 is satisfied, where $\mathrm{B}_1 \leftarrow \mathrm{B}_2$ indicates that B_1 is an ancestor of B_2. We denote the abbreviated valid PROPOSAL messages by $\langle \text{PROPOSAL}_1, ..., \mathrm{B}_1, qc_1 \rangle$ and $\langle \text{PROPOSAL}_2, ..., \mathrm{B}_2, qc_2 \rangle$, which are committed in views v_1 and v_2 respectively.

Lemma 1. *If B_1 and B_2 are conflicting blocks committed by honest replicas under ResiBFT, then $\mathrm{v}_1 \neq \mathrm{v}_2$.*

Proof. *We prove by contradiction, assuming that $\mathrm{v}_1 = \mathrm{v}_2$. We consider the following cases:* ***(a)*** *A valid block is committed after receiving at least $2f+1$ COMMIT messages in NORMAL stage or UNCIVIL stage, among which at least $f+1$ are cast by honest replicas for the correct proposal. If $\mathrm{v}_1 = \mathrm{v}_2$, suppose that f Byzantine replicas vote for both B_1 and B_2 by verifying qc_1 and qc_2. Then, there must exist at least one honest replica that also votes for both of them simultaneously;* ***(b)*** *Similarly, a valid block is committed after receiving at least $f+1$ accepted-vcs in GRACIOUS stage. If $\mathrm{v}_1 = \mathrm{v}_2$, suppose that f Byzantine replicas send accepted-vcs for both B_1 and B_2. Then, there must exist at least one honest replica that also sends an accepted-vc for both of them simultaneously. Both lead to a contradiction with the threat model.*

Theorem 2. (Safety of ResiBFT) *If B_1 and B_2 are conflicting blocks, they are never committed by honest replicas under ResiBFT.*

Proof. *We prove by contradiction, assuming that B_1 and B_2 are committed by honest replicas in v_1 and v_2. By Lemma 1, we have $\mathrm{v}_1 \neq \mathrm{v}_2$, and assume w.l.o.g. $\mathrm{v}_1 < \mathrm{v}_2$. We suppose the existence of v' and B'_1, satisfying $\mathrm{v}_1 < \mathrm{v}' < \mathrm{v}_2$, where B'_1 is the first block conflicting with B_1 proposed in v'. We consider the following cases:* ***(a)*** *Assume that an honest G-Replica $\mathcal{R}_1$ votes for B'_1 in v'. Let v'' denote the view of its locked block, satisfying $\mathrm{v}_1 \leq \mathrm{v}'' < \mathrm{v}'$. Let B'_2 be another proposal in v' satisfying $\mathrm{B}_1 \leftarrow \mathrm{B}'_2$. Then $\mathcal{R}_1$ must also vote for B'_2 to satisfy SAFENODE predicate. This implies that $\mathcal{R}_1$ voted for both B'_1 and B'_2 in v', leading to a contradiction with our threat model;* ***(b)*** *Similarly, assume that an honest T-Replica $\mathcal{R}_2$ votes for B'_1 in v'. Then it must also vote for B'_2 to satisfy the requirements of CHECKER, leading to a contradiction with our threat model. Therefore, honest replicas never commit conflicting blocks when $\mathrm{v}_1 < \mathrm{v}_2$. The same conclusion holds for the case when $\mathrm{v}_1 > \mathrm{v}_2$. In summary, honest replicas never commit conflicting blocks under ResiBFT.*

We assume that an adversary gains complete control over all f Byzantine replicas.

Theorem 3. (Liveness of ResiBFT) *After GST, if the leader is honest, all honest replicas eventually commit valid blocks under ResiBFT.*

Proof. *We consider the following cases:* ***(a)*** *In NORMAL stage and UNCIVIL stage, all replicas commit valid blocks through a 3-phase agreement process. If the leader is honest, it collects q_G NEWVIEW messages and multicasts a proposal extending from the highest block. Even if f Byzantine replicas vote incorrectly, at least $f+1$ honest replicas constitute a voting majority. All honest replicas complete the verification of each qc in the subsequent phases. After GST, all honest replicas eventually reach consensus and commit valid blocks;* ***(b)*** *In GRACIOUS stage, honest internal replicas submit the latest prepared blocks. If the leader is honest, it collects q_T NEWVIEW messages, multicasts a proposal extending from the highest block, and generate a qc for q_G vcs. Note that even if f Byzantine replicas generate vc maliciously, at least $f+1$ honest replicas constitute a voting majority. All honest replicas complete the verification of each qc in the subsequent phases. The leader then multicasts a bc for lightweight verification. TEE-compromised replicas and Byzantine T-Replicas may violate local safety or liveness. However, such malicious behavior can eventually be detected, and the corresponding view can still complete correctly. After GST, all honest replicas verify the bc and tentatively execute blocks. It will eventually be committed once the corresponding bc is checked.*

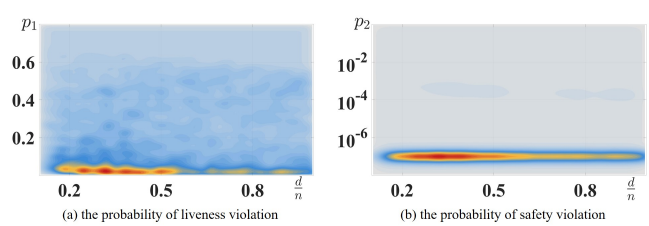
B. Local Safety and Liveness

We analyze the local safety and liveness violations from probabilistic perspective. In a committee $\{\mathcal{R}_C\}$, let t_1 denote the number of unresponsive replicas due to temporary crashes or Byzantine behavior, and t_2 the number of TEE-compromised replicas. Both serve as process variables in our analysis. Let p_1 and p_2 denote the probability of safety and liveness violation under ideal conditions, respectively. Here, 'ideal' means that t_1 and t_2 follow hypergeometric distributions, namely $t_1 \sim H(n, k, f)$ and $t_2 \sim H(n, k, \lambda f)$, while k is sampled from a uniform distribution, $k \sim U(\underline{k}, \min(d, 2f))$ [16]. We have $p_1 = 0$ and $p_2 = 0$ when $k \geq 2f+1$. Thus, the effective upper bound of k is $\min(d, 2f)$. $\underline{k}$ denotes the minimum estimated value that satisfies the committee fault tolerance requirement [38]. In our setting, we use $\underline{k} = 30$, as justified in Appendix A.

Specifically, p_1 and p_2 corresponds to the right-tail probability of a hypergeometric distribution, which is given by

$$\begin{cases} p_1 = P\left(t_1 \geq \lfloor (k+1)/2 \rfloor\right) \\ p_2 = P\left(t_2 \geq \lfloor (k+1)/2 \rfloor\right) \end{cases}. \tag{1}$$

We estimate the probability distributions of local safety and liveness violations through simulation under ideal conditions, as shown in Figure 6. In the simulation, the replica behavior is modeled as random. The horizontal axis d/n denotes the proportion of T-Replicas in the system. As this proportion increases, p_1 tends to decrease slightly, whereas p_2 remains consistently low (below 10^{-6}). Although p_1 is concentrated around a relatively low value (approximately 0.1), it exhibits substantial variability, with extreme cases reaching about 0.6. This suggests that without a colluding adversary, local safety can still be preserved, while local liveness relies on the epoch-level timer $\mathrm{A_e}$ for reliable recovery.



(a) the probability of liveness violation (b) the probability of safety violation

Fig. 6. Probability distribution of safety and liveness violations.

VIII. Experiments

IX. Conclusions

We propose ResiBFT, a hybrid BFT protocol for partial-TEE network systems. ResiBFT leverages TEE-based BFT protocols within the committee while incorporating conventional BFT protocols as a fallback, combining the strengths of both. We develop Hotsus, a prototype implementation of ResiBFT that extends HotStuff and Damysus. Through theoretical modeling and deployment experiments, we analyze the safety, liveness, and performance of both ResiBFT and Hotsus. Experimental results demonstrate that ResiBFT achieves system performance comparable to high-efficiency protocols, while requiring only $1/3$ of TEE-equipped replicas in the system.

References

[1] Y. Amir, B. Coan, J. Kirsch, and J. Lane, "Prime: Byzantine replication under attack," *IEEE transactions on dependable and secure computing*, vol. 8, no. 4, pp. 564–577, 2010.

[2] Y. Amir, C. Danilov, D. Dolev, J. Kirsch, J. Lane, C. Nita-Rotaru, J. Olsen, and D. Zage, "Steward: Scaling byzantine fault-tolerant replication to wide area networks," *IEEE Transactions on Dependable and Secure Computing*, vol. 7, no. 1, pp. 80–93, 2008.

[3] I. Anati, S. Gueron, S. Johnson, and V. Scarlata, "Innovative technology for cpu based attestation and sealing," in *Proceedings of the 2nd international workshop on hardware and architectural support for security and privacy*, vol. 13, no. 7. ACM New York, NY, USA, 2013.

[4] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich *et al.*, "Hyperledger fabric: a distributed operating system for permissioned blockchains," in *Proceedings of the thirteenth EuroSys conference*, 2018, pp. 1–15.

[5] A. M. Antonopoulos and G. Wood, *Mastering ethereum: building smart contracts and dapps*. O'reilly Media, 2018.

[6] J. Aspnes, "Randomized protocols for asynchronous consensus," *Distributed Computing*, vol. 16, no. 2, pp. 165–175, 2003.

[7] P.-L. Aublin, S. B. Mokhtar, and V. Quéma, "Rbft: Redundant byzantine fault tolerance," in *2013 IEEE 33rd international conference on distributed computing systems*. IEEE, 2013, pp. 297–306.

[8] J. Behl, T. Distler, and R. Kapitza, "Hybrids on steroids: Sgx-based high performance bft," in *Proceedings of the Twelfth European Conference on Computer Systems*, 2017, pp. 222–237.

[9] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the weil pairing," *Journal of cryptology*, vol. 17, pp. 297–319, 2004.

[10] E. Buchman, "Tendermint: Byzantine fault tolerance in the age of blockchains," Ph.D. dissertation, University of Guelph, 2016.

[11] C. Cachin, K. Kursawe, and V. Shoup, "Random oracles in constantipole: practical asynchronous byzantine agreement using cryptography," in *Proceedings of the nineteenth annual ACM symposium on Principles of distributed computing*, 2000, pp. 123–132.

[12] M. Castro and B. Liskov, "Proactive recovery in a {Byzantine-Fault-Tolerant} system," in *Fourth Symposium on Operating Systems Design and Implementation (OSDI 2000)*, 2000.

[13] M. Castro, B. Liskov *et al.*, "Practical byzantine fault tolerance," in *OsDI*, vol. 99, no. 1999, 1999, pp. 173–186.

[14] B. Y. Chan and E. Shi, "Streamlet: Textbook streamlined blockchains," in *Proceedings of the 2nd ACM Conference on Advances in Financial Technologies*, 2020, pp. 1–11.

[15] T. H. Chan, R. Pass, and E. Shi, "Pala: A simple partially synchronous blockchain," *Cryptology ePrint Archive*, 2018.

[16] X. Chen, S. Zhao, J. Qi, J. Jiang, H. Song, C. Wang, T. On Li, T. Hubert Chan, F. Zhang, X. Luo *et al.*, "Efficient and dos-resistant consensus for permissioned blockchains," *ACM SIGMETRICS Performance Evaluation Review*, vol. 49, no. 3, pp. 61–62, 2022.

[17] R. Cheng, F. Zhang, J. Kos, W. He, N. Hynes, N. Johnson, A. Juels, A. Miller, and D. Song, "Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts," in *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2019, pp. 185–200.

[18] J. Chuang, A. Seto, N. Berrios, S. van Schaik, C. Garman, and D. Genkin, "Tee. fail: Breaking trusted execution environments via ddr5 memory bus interposition," in *47th IEEE Symposium on Security and Privacy (IEEE S&P'26). IEEE Computer Society*, 2026.

[19] B.-G. Chun, P. Maniatis, S. Shenker, and J. Kubiatowicz, "Attested append-only memory: Making adversaries stick to their word," *ACM SIGOPS Operating Systems Review*, vol. 41, no. 6, pp. 189–204, 2007.

[20] A. Clement, M. Kapritsos, S. Lee, Y. Wang, L. Alvisi, M. Dahlin, and T. Riche, "Upright cluster services," in *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, 2009, pp. 277–290.

[21] A. Clement, E. Wong, L. Alvisi, M. Dahlin, M. Marchetti *et al.*, "Making byzantine fault tolerant systems tolerate byzantine faults," in *Proceedings of the 6th USENIX symposium on Networked systems design and implementation*. The USENIX Association, 2009, pp. 153–168.

[22] G. Coker, J. Guttman, P. Loscocco, A. Herzog, J. Millen, B. O'Hanlon, J. Ramsdell, A. Segall, J. Sheehy, and B. Sniffen, "Principles of remote attestation," *International journal of information security*, vol. 10, no. 2, pp. 63–81, 2011.

[23] M. Correia, N. F. Neves, and P. Verissimo, "How to tolerate half less one byzantine nodes in practical distributed systems," in *Proceedings of the 23rd IEEE International Symposium on Reliable Distributed Systems, 2004*. IEEE, 2004, pp. 174–183.

[24] V. Costan and S. Devadas, "Intel sgx explained," *Cryptology ePrint Archive*, 2016.

[25] J. De Meulemeester, D. Oswald, I. Verbauwhede, and J. Van Bulck, "Battering ram: Lowcost interposer attacks on confidential computing via dynamic memory aliasing," in *47th IEEE Symposium on Security and Privacy (S&P)*, 2026.

[26] J. Decouchant, D. Kozhaya, V. Rahli, and J. Yu, "Damysus: streamlined bft consensus leveraging trusted components," in *Proceedings of the Seventeenth European Conference on Computer Systems*, 2022, pp. 1–16.

[27] ——, "Oneshot: View-adapting streamlined bft protocols with trusted execution environments," in *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2024, pp. 1022–1033.

[28] ——, "Pallas and aegis: Rollback resilience in tee-aided blockchain consensus," in *33rd Annual Network and Distributed System Security Symposium*. The Internet Society, 2026, pp. 1–17.

[29] B. Dinis, P. Druschel, and R. Rodrigues, "Rr: A fault model for efficient tee replication," in *The Network and Distributed System Security Symposium*. Internet Society, 2023.

[30] T. Distler, "Byzantine fault-tolerant state-machine replication from a systems perspective," *ACM Computing Surveys (CSUR)*, vol. 54, no. 1, pp. 1–38, 2021.

[31] T. Distler, C. Cachin, and R. Kapitza, "Resource-efficient byzantine fault tolerance," *IEEE transactions on computers*, vol. 65, no. 9, pp. 2807–2819, 2015.

[32] T. Distler, I. Popov, W. Schröder-Preikschat, H. P. Reiser, and R. Kapitza, "Spare: Replicas on hold." in *NDSS*, 2011.

[33] C. Dwork, N. Lynch, and L. Stockmeyer, "Consensus in the presence of partial synchrony," *Journal of the ACM (JACM)*, vol. 35, no. 2, pp. 288–323, 1988.

[34] M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of distributed consensus with one faulty process," *Journal of the ACM (JACM)*, vol. 32, no. 2, pp. 374–382, 1985.

[35] M. Gao, H. Dang, E.-C. Chang, and J. Li, "Mixed fault tolerance protocols with trusted execution environment," *arXiv preprint arXiv:2208.01946*, 2022.

[36] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich, "Algorand: Scaling byzantine agreements for cryptocurrencies," in *Proceedings of the 26th symposium on operating systems principles*, 2017, pp. 51–68.

[37] S. Gueron, "A memory encryption engine suitable for general purpose processors," *Cryptology ePrint Archive*, 2016.

[38] G. G. Gueta, I. Abraham, S. Grossman, D. Malkhi, B. Pinkas, M. Reiter, D.-A. Seredinschi, O. Tamir, and A. Tomescu, "Sbft: A scalable and decentralized trust infrastructure," in *2019 49th Annual IEEE/IFIP international conference on dependable systems and networks (DSN)*. IEEE, 2019, pp. 568–580.

[39] L. J. Gunn, J. Liu, B. Vavala, and N. Asokan, "Making speculative bft resilient with trusted monotonic counters," in *2019 38th Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 2019, pp. 133–13 309.

[40] S. Gupta, S. Rahnama, S. Pandey, N. Crooks, and M. Sadoghi, "Dissecting bft consensus: In trusted components we trust!" in *Proceedings of*

the Eighteenth European Conference on Computer Systems, 2023, pp. 521–539.

[41] R. Kapitza, J. Behl, C. Cachin, T. Distler, S. Kuhnle, S. V. Mohammadi, W. Schröder-Preikschat, and K. Stengel, "Cheapbft: Resource-efficient byzantine fault tolerance," in *Proceedings of the 7th ACM european conference on Computer Systems*, 2012, pp. 295–308.

[42] D. Kaplan, J. Powell, and T. Woller, "Amd memory encryption," *White paper*, vol. 13, 2016.

[43] R. Kotla, L. Alvisi, M. Dahlin, A. Clement, and E. Wong, "Zyzzyva: speculative byzantine fault tolerance," in *Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles*, 2007, pp. 45–58.

[44] K. Kursawe, "Optimistic byzantine agreement," in *21st IEEE Symposium on Reliable Distributed Systems, 2002. Proceedings.* IEEE, 2002, pp. 262–267.

[45] L. Lamport, "The part-time parliament," *ACM Transactions on Computer Systems*, vol. 16, no. 2, pp. 133–169, 1998.

[46] L. Lamport, R. Shostak, and M. Pease, "The byzantine generals problem," *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982.

[47] D. Lee, D. Kohlbrenner, S. Shinde, K. Asanović, and D. Song, "Keystone: An open framework for architecting trusted execution environments," in *Proceedings of the Fifteenth European Conference on Computer Systems*, 2020, pp. 1–16.

[48] D. Levin, J. R. Douceur, J. R. Lorch, and T. Moscibroda, "Trinc: Small trusted hardware for large distributed systems." in *NSDI*, vol. 9, 2009, pp. 1–14.

[49] J. Li and D. Mazieres, "Beyond one-third faulty replicas in byzantine fault tolerant systems." in *NSDI*, 2007, pp. 10–10.

[50] J. Lind, O. Naor, I. Eyal, F. Kelbert, E. G. Sirer, and P. Pietzuch, "Teechain: a secure payment network with asynchronous blockchain access," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 2019, pp. 63–79.

[51] J. Liu, W. Li, G. O. Karame, and N. Asokan, "Scalable byzantine consensus via hardware-assisted secret sharing," *IEEE Transactions on Computers*, vol. 68, no. 1, pp. 139–151, 2018.

[52] S. Liu, P. Viotti, C. Cachin, V. Quéma, and M. Vukolić, "{XFT}: Practical fault tolerance beyond crashes," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 2016, pp. 485–500.

[53] S. Matetic, M. Ahmed, K. Kostiainen, A. Dhar, D. Sommer, A. Gervais, A. Juels, and S. Capkun, "{ROTE}: Rollback protection for trusted execution," in *26th USENIX Security Symposium (USENIX Security 17)*, 2017, pp. 1289–1306.

[54] I. Messadi, M. H. Becker, K. Bleeke, L. Jehl, S. B. Mokhtar, and R. Kapitza, "Splitbft: Improving byzantine fault tolerance safety using trusted compartments," in *Proceedings of the 23rd ACM/IFIP International Middleware Conference*, 2022, pp. 56–68.

[55] A. Miller, Y. Xia, K. Croman, E. Shi, and D. Song, "The honey badger of bft protocols," in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, 2016, pp. 31–42.

[56] N. Mohnblatt, A. Sonnino, K. Gurkan, and P. Jovanovic, "Arke: Scalable and byzantine fault tolerant privacy-preserving contact discovery," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 3212–3226.

[57] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," *Decentralized business review*, 2008.

[58] B. Ngabonziza, D. Martin, A. Bailey, H. Cho, and S. Martin, "Trustzone explained: Architectural features and use cases," in *2016 IEEE 2nd International Conference on Collaboration and Internet Computing (CIC)*. IEEE, 2016, pp. 445–451.

[59] R. Pass and E. Shi, "Hybrid consensus: Efficient consensus in the permissionless model," in *31st international symposium on distributed computing (DISC 2017)*. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2017, pp. 39–1.

[60] D. Porto, J. Leitão, C. Li, A. Clement, A. Kate, F. Junqueira, and R. Rodrigues, "Visigoth fault tolerance," in *Proceedings of the Tenth European Conference on Computer Systems*, 2015, pp. 1–14.

[61] H. Raj, S. Saroiu, A. Wolman, R. Aigner, J. Cox, P. England, C. Fenner, K. Kinshumann, J. Loeser, D. Mattoon *et al.*, "{fTPM}: A {Software-Only} implementation of a {TPM} chip," in *25th USENIX Security Symposium (USENIX Security 16)*, 2016, pp. 841–856.

[62] M. Russinovich, E. Ashton, C. Avanessians, M. Castro, A. Chamayou, S. Clebsch, M. Costa, C. Fournet, M. Kerner, S. Krishna *et al.*, "Ccf: A framework for building confidential verifiable replicated services," *Microsoft, Redmond, WA, USA, Tech. Rep. MSR-TR-2019-16*, 2019.

[63] F. B. Schneider, "Implementing fault-tolerant services using the state machine approach: A tutorial," *Acm Computing Surveys (CSUR)*, vol. 22, no. 4, pp. 299–319, 1990.

[64] A. Seto, O. K. Duran, S. Amer, J. Chuang, S. van Schaik, D. Genkin, and C. Garman, "Wiretap: Breaking server sgx via dram bus interposition," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025, pp. 708–722.

[65] A. Shamis, P. Pietzuch, B. Canakci, M. Castro, C. Fournet, E. Ashton, A. Chamayou, S. Clebsch, A. Delignat-Lavaud, M. Kerner *et al.*, "{IA-CCF}: Individual accountability for permissioned ledgers," in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 2022, pp. 467–491.

[66] M. Shih, S. Lee, T. Kim, and M. Peinado, "Eradicating controlled-channel attacks against enclave programs," *NDSS, T-SGX*, 2017.

[67] V. Shoup, "Practical threshold signatures," in *Advances in Cryptology—EUROCRYPT 2000: International Conference on the Theory and Application of Cryptographic Techniques Bruges, Belgium, May 14–18, 2000 Proceedings 19*. Springer, 2000, pp. 207–220.

[68] P. Thambidurai and Y.-K. Park, "Interactive consistency with multiple failure modes," in *Proceedings [1988] Seventh Symposium on Reliable Distributed Systems*. IEEE, 1988, pp. 93–100.

[69] L. Tian, C. Wang, Y. Long, X. Xu, M. Wan, C. Li, S.-F. Sun, and D. Gu, "Walnut: A generic framework with enhanced scalability for bft protocols," in *Australasian Conference on Information Security and Privacy*. Springer, 2025, pp. 351–371.

[70] P. E. Veríssimo, "Travelling through wormholes: a new look at distributed systems models," *ACM SIGACT News*, vol. 37, no. 1, pp. 66–81, 2006.

[71] G. S. Veronese, M. Correia, A. N. Bessani, and L. C. Lung, "Spin one's wheels? byzantine fault tolerance with a spinning primary," in *2009 28th IEEE International Symposium on Reliable Distributed Systems*. IEEE, 2009, pp. 135–144.

[72] ——, "Ebawa: Efficient byzantine agreement for wide-area networks," in *2010 IEEE 12th International Symposium on High Assurance Systems Engineering*. IEEE, 2010, pp. 10–19.

[73] G. S. Veronese, M. Correia, A. N. Bessani, L. C. Lung, and P. Verissimo, "Efficient byzantine fault-tolerance," *IEEE Transactions on Computers*, vol. 62, no. 1, pp. 16–30, 2011.

[74] C. Wang, S. Duan, M. Xu, F. Li, and X. Cheng, "Consensus in the known participation model with byzantine failures and sleepy replicas," *Cryptology ePrint Archive*, 2024.

[75] G. Wang and M. Nixon, "Sok: X-assisted bft consensus protocols," in *International Conference on Blockchain*. Springer, 2023, pp. 54–71.

[76] R. Wang, A. Vogel, and P.-L. Aublin, "Bftaas: Byzantine fault tolerance as a service through nic-level aggregation," in *2025 IEEE Conference on Dependable, Autonomic and Secure Computing (DASC)*. IEEE, 2025, pp. 141–143.

[77] W. Wang, S. Deng, J. Niu, M. K. Reiter, and Y. Zhang, "Engraft: Enclave-guarded raft on byzantine faulty nodes," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 2841–2855.

[78] X. Wang, H. Wang, H. Zhang, and S. Duan, "Pando: Extremely scalable bft based on committee sampling." *IACR Cryptol. ePrint Arch.*, vol. 2024, p. 664, 2024.

[79] G. Wood *et al.*, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum project yellow paper*, vol. 151, no. 2014, pp. 1–32, 2014.

[80] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, "Hotstuff: Bft consensus with linearity and responsiveness," in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, 2019, pp. 347–356.

[81] J. Zhang, J. Gao, K. Wang, Z. Wu, Y. Li, Z. Guan, and Z. Chen, "Tbft: efficient byzantine fault tolerance using trusted execution environment," in *ICC 2022-IEEE International Conference on Communications*. IEEE, 2022, pp. 1004–1009.

[82] L. Zhao, J. Decouchant, J. K. Liu, Q. Lu, and J. Yu, "Trusted hardware-assisted leaderless byzantine fault tolerance consensus," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 6, pp. 5086–5097, 2024.

APPENDIX

A. Theoretical proofs

Theorem 4. *The committee-based design entails an inherent safety–liveness trade-off — a higher safety guarantee comes at the cost of lower liveness, and vice versa.*

Proof. Let p_1 and p_2 denote the probability of safety and liveness violation under ideal conditions, respectively. The committee violate the safety when the internal replicas no longer satisfy the TEE-based protocol assumption $2t+1 \leq d$, where t denotes the number of unresponsive replicas in the committee (see Section V-B). Here, 'ideal conditions' mean that t follows a hypergeometric distribution defined by Equation 2, $t \sim H(n, d, f)$, and d follows a uniform distribution, $d \sim U(\underline{d}, \min(D, 2f))$ [16], where $\underline{d}$ denotes the lowest estimated value that satisfies the committee fault tolerance requirement for providing security guarantees [38] (see Section **??**). When $d \geq 2f+1$, we have $p = 0$, in which case HyTEE-BFT effectively reduces to Damysus.

$$P(t = X) = \frac{\binom{d}{X}\binom{n-d}{f-X}}{\binom{n}{f}}. \tag{2}$$

Thus, p corresponds to the right-tail probability of a hypergeometric distribution, which is given by

$$p = P(t \geq \lfloor (d-1)/2 \rfloor) = 1 - \sum_{t=0}^{\lfloor (d-1)/2 \rfloor} P(t = X). \tag{3}$$

We have $p = 0$ when $d > 2f+1$. When $d \leq 2f+1$, safety can be established using the Chernoff upper tail bound. Let $\mu = f \cdot \frac{d}{n} = \Theta(d)$, and in particular $\mu \leq d/3$. The target probability p corresponds to the event that t exceeds μ by a linear margin, i.e., $t \geq (1+\delta)\mu$ for some constant $\delta > 0$ (e.g., $\delta \approx 1/2$). Since hypergeometric random variables satisfy Chernoff bound, we have

$$p = P(t \geq \lfloor (d-1)/2 \rfloor) \leq \exp(-\Theta(\mu)) = \exp(-\Theta(d)). \tag{4}$$

Thus, the probability of committee failures p decreases exponentially with d. Our calculation shows that an acceptable committee failures probability of $p < 5\%$ is achieved when $d > 20$. Accordingly, we set $\underline{d} = 21$ in our paper.